

Monotonic Stack

Description of the Data Structure

A monotonic stack is a stack that maintains its elements in a monotonic order. My implementation uses a stack structure, where each Block stores the index of a gate and a pointer to the next block. The top pointer represents the top of the stack, so insertion and removal are performed only at the top. In this problem, the stack stores the indices of gates that are still potential blockers. Before a new gate is pushed, gates with lower or equal clearance levels are removed if they are dominated by the new gate. Subsequently, keycards can also remove gates from the top when their clearance is sufficient to open them. Therefore, the stack only retains gates that may still prevent traversal from the current or a later room.

Why?

It is a monotonic stack because the clearance levels of the stored gates are maintained in strictly decreasing order from the bottom to the top. Before a new gate is pushed, all gates at the top with clearance less than or equal to the new gate are removed. This preserves the monotonic invariant and removes gates that can no longer be the nearest relevant blocker. A monotonic queue could similarly maintain elements in monotonic order, but it would require operations at both ends. For this problem, only the most recent relevant blocker needs to be examined and removed, so the LIFO behaviour of a stack is sufficient and simpler.

Complexity

A single monotonic-stack operation can take $O(p)$ in the worst case because one insertion may cause many existing blocks to be popped. However, across p processed elements, each block is pushed onto the stack exactly once and can be popped at most once. Once a block is popped, it is permanently removed and cannot contribute to the cost of a later operation. Therefore, across all operations there are at most p pushes and p pops. The total number of stack operations is at most $2p$, giving a time complexity of $O(p)$.

Testing Evidence

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
8
Enter gates:
5
2
3
2
2
3
1
2
Enter keycards:
1
4
1
1
3
2
0
2
0 1 0 0 4 0 0 2
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
1
Enter gates:
5
Enter keycards:
5
1
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
1
Enter gates:
5
Enter keycards:
1
0
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
4
Enter gates:
5
4
3
2
Enter keycards:
0
0
0
0
0 0 0 0
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
4
Enter gates:
5
4
3
2
Enter keycards:
10
10
10
10
1 2 3 4
```

```
yihaoyu@dyn-118-138-78-77 HD1 % ./main test
Enter n:
3
Enter gates:
8
5
2
Enter keycards:
1
6
3
0 1 2
```